28.3 混合搜索（BM25 + 向量）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~330k tokens，输出 ~7k tokens（本节）

向量搜索擅长捕捉**语义相关性**——"TypeScript 配置" 和 "TS 项目设置" 的向量距离很近，即使它们没有共享关键词。但向量搜索也有盲区：当用户搜索精确的函数名 `calculateTotal` 时，传统的关键词匹配反而更精准。OpenClaw 的解决方案是**混合搜索（Hybrid Search）**——将 BM25 全文搜索与向量搜索的结果融合，兼顾精确匹配和语义理解。

28.3.1 BM25 全文搜索原理

什么是 BM25?

> **衍生解释**：BM25（Best Matching 25）是信息检索领域最经典的排名算法之一，由 Stephen Robertson 等人在 1994 年提出。它是 TF-IDF（词频-逆文档频率）的改进版，核心思想是：一个词在某文档中出现的频率越高（TF），在整个文档集中出现的频率越低（IDF），这个词对该文档的重要性就越大。BM25 在 TF-IDF 基础上增加了两个改进：(1) 词频饱和——词频增长到一定程度后，贡献不再线性增加；(2) 文档长度归一化——长文档不会因为词多而占优势。公式为：
>
> $$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t,d) \cdot (k_1 + 1)}{f(t,d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\mathrm{avgdl}})}$$
>
> 其中 $f(t,d)$ 是词 t 在文档 d 中的频率，$|d|$ 是文档长度，avgdl 是平均文档长度，k_1 和 b 是可调参数。

OpenClaw 不需要自己实现 BM25——SQLite 的 FTS5 引擎内置了 BM25 排名函数。OpenClaw 要做的是：(1) 将文本块插入 FTS5 虚拟表，(2) 构造合适的 MATCH 查询，(3) 将 BM25 排名转换为可与向量分数融合的归一化分数。

FTS 查询构造

用户的自然语言查询需要转换为 FTS5 能理解的 MATCH 表达式。 `buildFtsQuery` 负责这个转换：

```
// src/memory/hybrid.ts

export function buildFtsQuery(raw: string): string | null {
  const tokens = raw
    .match(/[A-Za-z0-9_]+/g)           // 提取所有字母数字词
    ?.map(t => t.trim())
    .filter(Boolean) ?? [];
  if (tokens.length === 0) return null;  // 没有有效词，放弃搜索
  const quoted = tokens.map(t => `"${t.replaceAll('"', '')}"`);
  return quoted.join(" AND ");           // 所有词必须同时出现
}
```

转换示例：

用户输入	FTS 查询	含义
`TypeScript 配置`	`"TypeScript" AND "配置"`	— 注：中文被过滤，只剩 `"TypeScript"`
`calculate total price`	`"calculate" AND "total" AND "price"`	三个词必须全部出现
`API_KEY setup`	`"API_KEY" AND "setup"`	下划线视为词的一部分
你好世界	`null`	全是中文，无有效 token

注意一个重要限制：`buildFtsQuery` 使用 `[A-Za-z0-9_]+` 正则提取 token，这意味着**纯中文查询会被过滤为空**，回退到纯向量搜索。这是一个合理的权衡——FTS5 默认的分词器对中文支持有限，而向量搜索天然支持多语言。

关键词搜索实现

`searchKeyword` 执行实际的 FTS5 查询：

```ts
// src/memory/manager-search.ts（简化）

export async function searchKeyword(params: {
  db: DatabaseSync;
  ftsTable: string;          // "chunks_fts"
  providerModel: string;
  query: string;
  limit: number;
  snippetMaxChars: number;
  sourceFilter: { sql: string; params: string[] };
  buildFtsQuery: (raw: string) => string | null;
  bm25RankToScore: (rank: number) => number;
}): Promise<Array<SearchRowResult & { textScore: number }>> {
  const ftsQuery = params.buildFtsQuery(params.query);
  if (!ftsQuery) return [];  // 无有效查询词

  const rows = params.db.prepare(
    `SELECT id, path, source, start_line, end_line, text,
            bm25(${params.ftsTable}) AS rank
       FROM ${params.ftsTable}
      WHERE ${params.ftsTable} MATCH ?
        AND model = ?
      ORDER BY rank ASC
      LIMIT ?`
  ).all(ftsQuery, params.providerModel, params.limit);

  return rows.map(row => ({
    id: row.id,
    path: row.path,
    startLine: row.start_line,
    endLine: row.end_line,
    score: params.bm25RankToScore(row.rank),
    textScore: params.bm25RankToScore(row.rank),
    snippet: truncateUtf16Safe(row.text, params.snippetMaxChars),
    source: row.source,
  }));
}
```

几个要点：

1. **`bm25()` 函数**：FTS5 内置的排名函数，返回一个负数（越小越相关）。SQLite 文档中称其为 "rank"，值域为 $(-\infty, 0]$。

2. **`ORDER BY rank ASC`**：因为 rank 越小越好，所以升序排列。
3. **`model` 过滤**：确保只搜索当前嵌入模型对应的 chunks，避免跨模型数据混淆。

BM25 分数归一化

FTS5 的 `bm25()` 返回的是负数 rank，需要转换为 $[0, 1]$ 区间的分数才能与向量搜索的余弦相似度融合：

```
// src/memory/hybrid.ts

export function bm25RankToScore(rank: number): number {
  const normalized = Number.isFinite(rank) ? Math.max(0, rank) : 999;
  return 1 / (1 + normalized);
}
```

等等——`Math.max(0, rank)` 对负数 rank 会返回 0，而 `1 / (1 + 0) = 1`？实际上，SQLite FTS5 的 `bm25()` 返回值在 OpenClaw 的使用场景中会被 `ORDER BY rank ASC` 处理后传入，且 FTS5 的 rank 值经过 `bm25()` 聚合函数后在大多数实现中是**非负**的（具体取决于 SQLite 版本和配置）。转换函数使用倒数映射 $s = \frac{1}{1+r}$：

rank	score	含义
0	1.0	最相关
1	0.5	
4	0.2	
9	0.1	
999	≈ 0.001	几乎无关（或异常值）

这个映射是**单调递减**的，且自然地将分数压缩到 $(0, 1]$ 区间，与余弦相似度的值域 $[-1, 1]$（实际中通常为 $[0, 1]$）兼容。

28.3.2 混合检索实现

搜索入口

`MemoryIndexManager.search()` 是混合搜索的入口方法。它编排了三个阶段：关键词搜索、向量搜索、结果融合。

```ts
// src/memory/manager.ts（简化）

async search(query: string, opts?: { maxResults?: number; minScore?: number }) {
  const minScore = opts?.minScore ?? this.settings.query.minScore;
  const maxResults = opts?.maxResults ?? this.settings.query.maxResults;
  const hybrid = this.settings.query.hybrid;

  // 候选数量 = 最终结果数 × 候选倍数（上限 200）
  const candidates = Math.min(200,
    Math.max(1, Math.floor(maxResults * hybrid.candidateMultiplier))
  );

  // 阶段 1：关键词搜索（BM25）
  const keywordResults = hybrid.enabled
    ? await this.searchKeyword(query, candidates).catch(() => [])
    : [];

  // 阶段 2：向量搜索
  const queryVec = await this.embedQueryWithTimeout(query);
  const hasVector = queryVec.some(v => v !== 0);
  const vectorResults = hasVector
    ? await this.searchVector(queryVec, candidates).catch(() => [])
    : [];

  // 阶段 3：融合或直接返回
  if (!hybrid.enabled) {
    return vectorResults
      .filter(e => e.score >= minScore)
      .slice(0, maxResults);
  }

  const merged = this.mergeHybridResults({
    vector: vectorResults,
    keyword: keywordResults,
    vectorWeight: hybrid.vectorWeight,    // 默认 0.7
    textWeight: hybrid.textWeight,        // 默认 0.3
  });

  return merged
    .filter(e => e.score >= minScore)
    .slice(0, maxResults);
}
```

关键设计决策：

1. **候选倍数（candidateMultiplier）**：如果用户只需要 10 条结果，系统会向两个搜索通道分别请求 `10 × multiplier` 条候选（默认 multiplier = 3，即 30 条）。更多候选意味着融合后的 Top-K 更准确，代价是搜索量略大。
2. **错误隔离**：两个搜索通道分别用 `.catch(() => [])` 包裹——关键词搜索失败不影响向量搜索，反之亦然。
3. **零向量检测**：如果嵌入 API 返回全零向量（通常意味着嵌入失败），则跳过向量搜索，只使用关键词结果。

用时序图表示完整搜索流程：

```
用户查询 "TypeScript 项目配置"
         │
         ▼
  ┌──────────────────┐
  │ search()         │
  └──────────────────┘
         │
     ┌───┴───┐
     ▼       ▼
关键词通道   向量通道
    │       │
    │  buildFtsQuery()
    │  → "TypeScript"
    │       │
    │  embedQuery()
    │  → [0.023, -0.017, ...]
    │       │
    ▼       ▼
FTS5 MATCH  vec_distance_cosine
bm25() rank  余弦距离
    │       │
    ▼       ▼
bm25RankToScore  1 - distance
    │       │
    └───┬───┘
        ▼
  mergeHybridResults()
  ┌──────────────────────────┐
  │ vectorWeight × vecScore  │
  │ + textWeight × textScore │
  └──────────────────────────┘
             ▼
      排序 + 过滤 + 截断
             │
             ▼
      返回 MemorySearchResult[]
```

28.3.3 分数融合策略：加权线性组合

融合算法

`mergeHybridResults` 是混合搜索的核心——将两个来源的结果按 ID 合并，计算加权分数：

```typescript
// src/memory/hybrid.ts

export function mergeHybridResults(params: {
  vector: HybridVectorResult[];    // { id, vectorScore, ... }
  keyword: HybridKeywordResult[];  // { id, textScore, ... }
  vectorWeight: number;            // 默认 0.7
  textWeight: number;              // 默认 0.3
}): Array<{ path, startLine, endLine, score, snippet, source }> {
  const byId = new Map();

  // 1. 先收集向量搜索结果
  for (const r of params.vector) {
    byId.set(r.id, {
      ...r,
      vectorScore: r.vectorScore,
      textScore: 0,               // 默认文本分数为 0
    });
  }

  // 2. 合并关键词搜索结果
  for (const r of params.keyword) {
    const existing = byId.get(r.id);
    if (existing) {
      existing.textScore = r.textScore;  // 同一 chunk 同时命中
    } else {
      byId.set(r.id, {                   // 只在关键词中命中
        ...r,
        vectorScore: 0,
        textScore: r.textScore,
      });
    }
  }

  // 3. 计算加权分数并排序
  const merged = Array.from(byId.values()).map(entry => ({
    ...entry,
    score: params.vectorWeight * entry.vectorScore
         + params.textWeight  * entry.textScore,
  }));

  return merged.toSorted((a, b) => b.score - a.score);
}
```

融合公式非常直观：

$$\text{score}_{\text{final}} = w_v \times s_{\text{vector}} + w_t \times s_{\text{text}}$$

其中默认 $w_v = 0.7$，$w_t = 0.3$。

三种命中情况

一个文本块在融合时可能处于三种状态：

情况	向量分数	文本分数	最终分数
双重命中	0.85	0.72	$0.7 \times 0.85 + 0.3 \times 0.72 = 0.811$
仅向量命中	0.85	0	$0.7 \times 0.85 = 0.595$
仅关键词命中	0	0.72	$0.3 \times 0.72 = 0.216$

双重命中的 chunk 获得最高分数——这正是混合搜索的价值所在。一个既在语义上相关、又包含精确关键词的结果，几乎肯定是用户想要的。

权重的含义

默认的 7:3 权重偏向向量搜索，反映了一个设计判断：对于 AI Agent 的记忆检索场景，**语义理解比精确匹配更重要**。用户搜索 "之前讨论的数据库方案" 时，向量搜索能找到包含 "PostgreSQL vs MongoDB 对比" 的记忆块，而关键词搜索可能一无所获（因为没有共享词汇）。

但关键词搜索的 30% 权重也不可忽视——它在以下场景中起到关键补充：

- **精确名称搜索**：函数名、变量名、项目名
- **错误信息搜索**：搜索特定的报错文本
- **版本号搜索**：搜索 "v2.3.1" 这类精确字符串
- **中英混合查询**：英文关键词通过 BM25 精确匹配，中文语义通过向量补充

配置可调

权重和其他混合搜索参数都可以通过配置覆盖：

```
// src/agents/memory-search.ts（默认值）

const DEFAULT_HYBRID_VECTOR_WEIGHT = 0.7;
const DEFAULT_HYBRID_TEXT_WEIGHT = 0.3;
const DEFAULT_HYBRID_CANDIDATE_MULTIPLIER = 3;
const DEFAULT_HYBRID_ENABLED = true;
```

用户可以在 `settings.json` 中调整：

```
{
  "agents": {
    "defaults": {
      "memorySearch": {
        "hybrid": {
          "enabled": true,
          "vectorWeight": 0.5,
          "textWeight": 0.5,
          "candidateMultiplier": 5
        }
      }
    }
  }
}
```

设置 `vectorWeight: 0, textWeight: 1` 可以完全禁用向量搜索，只用 BM25；反过来设 `enabled: false` 则完全禁用混合搜索，只用向量。

28.3.4 从理论到实践：一次完整搜索

让我们跟踪一次完整的混合搜索，理解数据如何在各层流动。

场景：用户问 Agent "我们之前决定用什么数据库？"

步骤 1：Agent 调用 `memory_search` 工具，传入查询 "之前决定用什么数据库"

步骤 2：`search()` 方法启动两个并行通道

- **关键词通道**：`buildFtsQuery("之前决定用什么数据库")` → 所有中文被过滤 → 返回 `null` → 关键词搜索跳过

- **向量通道**：嵌入 API 将查询转为 1536 维向量 → sqlite-vec 执行余弦距离搜索 → 返回 Top-30 候选

步骤 3：由于关键词结果为空，融合退化为纯向量搜索

步骤 4：过滤 `score >= minScore` ，返回 Top-10 给 Agent

现在换一个场景：用户搜索 "PostgreSQL migration plan"

- **关键词通道**： `buildFtsQuery(...)` → `"PostgreSQL" AND "migration" AND "plan"` → BM25 返回包含这三个词的 chunks
- **向量通道**：向量搜索返回语义相关的 chunks（可能包含 "数据库迁移方案"、"从 MongoDB 切换"等）
- **融合**：同时命中的 chunks 得到最高分，如某个 chunk 标题是 "PostgreSQL migration plan — 2026.01.10"

这个例子很好地展示了混合搜索的互补性：关键词搜索精准锁定包含 "PostgreSQL" 的记忆，向量搜索补充了语义相关但措辞不同的结果。

本节小结

1. **BM25 全文搜索**通过 SQLite FTS5 实现， `buildFtsQuery` 将用户输入转换为 AND 连接的引号词查询， `bm25RankToScore` 将排名转换为 $(0, 1]$ 区间的归一化分数。
2. **混合搜索入口** `search()` 同时启动关键词和向量两个通道，各自独立容错（ `.catch(() => [])` ），通过候选倍数（默认 3x）确保融合质量。
3. **加权线性组合**是融合策略的核心：$\text{score} = 0.7 \times s_{\text{vec}} + 0.3 \times s_{\text{text}}$。双重命中的 chunk 获得最高分数，确保既语义相关又包含精确关键词的结果排在最前。
4. **中文查询的特殊处理**：由于 FTS5 分词器对中文支持有限，纯中文查询自然降级为纯向量搜索，这是一个合理的设计权衡。
5. **所有参数可配置**：权重比例、候选倍数、是否启用混合搜索，都可以通过 `settings.json` 调整，适应不同使用场景。